Phase 2: Core

LangChain Mastery — Study Guide

> **Where you are:** Phase 2 builds directly on Phase 1 (Setup, Models, Prompts, Output Parsers). By now you should be comfortable calling a chat model, building a prompt template, and parsing the result into a string or JSON object. Phase 2 is where LangChain stops being "a few helper functions" and starts being a real **framework** — you'll learn the connective tissue (Runnables/LCEL) that holds everything together, how to compose multi-step pipelines (Chains), and how to make your application remember things (Memory).
>
> This phase contains three units:
>
> - **Unit 04 — Runnables & LCEL** (the foundation everything else is built on)
> - **Unit 05 — Chains** (composing Runnables into real pipelines)
> - **Unit 06 — Memory** (giving your chains and chatbots a "remembered" conversation)

Unit 04 — Runnables & LCEL

Phase: Core | **Difficulty:** Intermediate

Why this unit matters

Every single thing you build in LangChain — a prompt, a model, a parser, a retriever, an agent — is secretly the same kind of object underneath: a **Runnable**. Once you understand Runnables, you understand the "grammar" of the entire LangChain library. Everything else (Chains, RAG pipelines, Agents) is just Runnables wired together in different shapes. This is the single most important unit in the whole roadmap — take your time here.

4.1 Concept and Theory

4.1.1 What is a Runnable?

The simple explanation: A Runnable is LangChain's word for "a unit of work that takes an input and produces an output, in a standard, predictable way."

Think about electrical appliances in your house. A toaster, a phone charger, and a lamp are all completely different devices that do completely different things — but they all share one thing in common: a standard plug. Because they all use the same plug shape, you can connect any of them to any wall socket in your house without thinking about the wiring underneath.

A **Runnable** is LangChain's "standard plug." A prompt template, a chat model, an output parser, a

retriever — they are all wildly different tools internally, but they all expose the *same* three methods:

Method	What it does	Real-world analogy
`invoke(input)`	Runs once, on one input, and gives you back one output	Pressing "start" on a single load of laundry
`batch([input1, input2, ...])`	Runs the same operation on a list of inputs, in parallel where possible	Running 5 loads of laundry at once across 5 machines
`stream(input)`	Runs once, but gives you the output piece-by-piece as it's produced, instead of waiting for everything to finish	Watching a kettle fill a cup gradually instead of waiting for the whole kettle to boil and then pouring it all at once

Why does this exist? Before this standard interface existed, every component in a chain might have a slightly different way of being called, which made it hard to mix and match pieces. By forcing every component to support `invoke`/`batch`/`stream`, LangChain lets you connect *any* component to *any* other component, the same way standard plugs let you connect any appliance to any wall socket. This is the entire reason "LCEL" (LangChain Expression Language) works — it's a generic system for wiring Runnables together because they all speak the same language.

When is this used? Constantly — every time you write `model.invoke(...)`, `prompt.invoke(...)`, or chain things together with `.pipe()`, you are using the Runnable interface, whether you realize it or not.

4.1.2 RunnableSequence — chaining steps one after another

The simple explanation: A `RunnableSequence` takes several Runnables and runs them **one after the other**, automatically feeding the output of step 1 as the input to step 2, the output of step 2 as the input to step 3, and so on.

Analogy: Think of an assembly line in a factory. Raw metal goes in at station 1 (cutting), the cut metal moves to station 2 (welding), the welded piece moves to station 3 (painting), and a finished product comes out the other end. You never have to manually carry the metal between stations — the conveyor belt does it. `RunnableSequence` *is* the conveyor belt connecting your prompt template, your model, and your parser.

Why does it exist? Almost every LLM task follows the same shape: *format a prompt → send it to a model → clean up the model's response.* Instead of you manually writing `const formattedPrompt = await prompt.invoke(input); const modelResponse = await model.invoke(formattedPrompt); const finalAnswer = await parser.invoke(modelResponse);` every single time, `RunnableSequence` lets you declare the *shape* of the pipeline once, and it handles passing data through automatically.

How it works: You can build one in two equivalent ways:

```
// Explicit constructor
const chain = RunnableSequence.from([prompt, model, parser]);

// Shorthand using pipe() — functionally identical
const chain = prompt.pipe(model).pipe(parser);
```

Both produce a single Runnable. That means the *whole chain itself* also has `invoke`, `batch`, and `stream` — chains are Runnables made of Runnables, all the way down. This is why LangChain composition feels like building with Lego bricks: any finished assembly can become a single brick inside a bigger assembly.

4.1.3 RunnableParallel — executing steps concurrently

The simple explanation: `RunnableParallel` takes the **same input** and sends it to *multiple* Runnables **at the same time**, then collects all of their outputs into a single object.

Analogy: Imagine you walk into a hospital for a check-up. Instead of seeing the blood-test technician, then waiting for them to finish before seeing the X-ray technician, then waiting again before seeing the heart specialist — a well-run hospital sends you to all three *at once* (different rooms, same building), and combines the three reports into one file at the end. `RunnableParallel` is that efficient hospital.

Why does it exist? If you need two unrelated outputs from the same input — for example, "give me a *summary*" AND "give me 5 *keywords*" from the same article — there's no reason to wait for the summary to finish before starting on the keywords. Running them in parallel can be significantly faster, since you're not paying the "waiting" cost twice.

How it works:

```
const mapChain = RunnableParallel.from({
  summary: summaryChain,
  keywords: keywordChain,
});
// Input goes to BOTH chains simultaneously.
// Output looks like: { summary: "...", keywords: "..." }
```

Notice the output is a plain object whose keys match the names you gave each branch. This is extremely useful as a "fan-out" step inside a bigger `RunnableSequence`.

4.1.4 RunnableBranch — conditional routing between runnables

The simple explanation: `RunnableBranch` looks at the input, checks it against a series of conditions (in order), and sends the input down whichever path's condition is `true` first. If none match, it falls back to a default path.

Analogy: Think of a hospital's emergency triage desk. The nurse looks at each patient and asks a series of yes/no questions: "Is this chest pain?" → send to cardiology. "Is this a broken bone?" → send to orthopedics. "None of the above?" → send to general practice. `RunnableBranch` is that triage nurse for your data.

Why does it exist? Real applications rarely use one single prompt for everything. A customer-support bot might need a different chain for "billing questions" versus "technical questions" versus "general questions." `RunnableBranch` lets you build *one* logical pipeline that internally decides which specialized sub-chain should actually handle the request.

How it works:

```
const branch = RunnableBranch.from([
  [(input) => input.topic === "billing", billingChain],
```

```
  [(input) => input.topic === "technical", technicalChain],
  generalChain, // default / fallback, no condition needed
]);
```

Each condition function receives the input and returns `true` or `false`. The branch is checked top-to-bottom, and the **first** match wins — order matters!

4.1.5 RunnableLambda — wrapping custom functions as Runnables

The simple explanation: `RunnableLambda` takes any plain JavaScript/TypeScript function you've written and "promotes" it into a full Runnable, so it can be plugged into a chain alongside prompts, models, and parsers.

Analogy: Imagine your assembly line (the `RunnableSequence`) needs a custom step that doesn't exist as an off-the-shelf machine — say, a very specific "stamp today's date onto this part" step. Instead of buying a new branded machine, you build a small custom jig yourself and bolt it onto the conveyor belt. `RunnableLambda` is that custom jig — it lets *your own logic* sit on the conveyor belt right alongside LangChain's built-in pieces.

Why does it exist? Not every step in a pipeline is "ask an LLM something." Sometimes you need to do plain old programming — trim whitespace, look something up in a database, do basic math, reformat a date. `RunnableLambda` is the bridge between "LangChain world" and "ordinary code."

How it works:

```
const upperCase = RunnableLambda.from((input) => input.toUpperCase());

const chain = prompt.pipe(model).pipe(parser).pipe(upperCase);
```

Anywhere a Runnable is expected, you can drop in a `RunnableLambda` instead, and the rest of the chain won't know (or care) that a piece of plain code ran instead of an LLM call.

4.1.6 RunnablePassthrough — forwarding input unchanged downstream

The simple explanation: `RunnablePassthrough` does *nothing* to its input — it just hands it straight through, unchanged, to the next step. That sounds useless at first, but it solves a very common, very specific problem.

Analogy: Imagine a relay race where you need the *original* baton (the original question) to still be available at the finish line, even though several runners (processing steps) have passed it between themselves and transformed copies of it along the way. `RunnablePassthrough` is like photocopying the baton at the start so the original is still around at the end, untouched.

Why does it exist? This shows up constantly in RAG (Retrieval-Augmented Generation, which you'll meet in Phase 3): you take the user's original question, use it to retrieve relevant documents, but you *still need the original question text* later to actually build the final prompt ("Answer {question} using this context: {context}"). Without `RunnablePassthrough`, the original question would get "lost" after being transformed into search results.

How it works (commonly paired with `RunnableParallel`):

```
const chain = RunnableParallel.from({
```

```
  context: retriever,               // transforms the question into documents
  question: new RunnablePassthrough(), // keeps the original question untouched
})
  .pipe(promptTemplate)
  .pipe(model);
```

Here, the same input (the user's question) is sent to *both* branches: one branch transforms it into retrieved context, the other branch just hands it through as-is. The result is an object `{ context: [...documents], question: "..." }` that the next prompt template can use.

4.1.7 pipe() syntax — shorthand for RunnableSequence

The simple explanation: `.pipe()` is just a more readable, chainable way to write `RunnableSequence.from([...])`. `a.pipe(b)` means "run `a`, then feed its output into `b`."

Why does it exist? Readability. Compare:

```
// Verbose
const chain = RunnableSequence.from([prompt, model, parser]);

// Piped — reads left-to-right like a sentence: "prompt, then model, then
parser"
const chain = prompt.pipe(model).pipe(parser);
```

Both produce *exactly* the same object. `.pipe()` is just syntactic sugar that most real-world LangChain.js code uses because it visually mirrors the flow of data.

4.2 Practical Lab

> **Before you start:** This lab assumes you completed Unit 00 (Setup) and already have a working Node.js project with `@langchain/core`, `@langchain/openai` (or another provider) installed, and an API key loaded via `.env`. If you haven't done that yet, go back to Unit 00 first — every lab in this guide builds on that base project.

Lab Setup

1. **Create a new file** inside your project, e.g. `unit04-runnables.js` (or `.ts` if you're using TypeScript). *Why:* Keeping each unit's experiments in its own file means you can re-run any lab independently later without your code getting tangled together.
2. **Add the imports you'll need for this whole lab at the top of the file:**

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import {
  RunnableSequence,
  RunnableParallel,
  RunnableBranch,
  RunnableLambda,
  RunnablePassthrough,
} from "@langchain/core/runnables";
```

What this does: `"dotenv/config"` loads your `.env` file's variables (like your API key) into `process.env` automatically. The rest are the building blocks you read about above. *Expected result:* No errors when you run the file (it won't print anything yet because we haven't called anything). *Common beginner mistake:* Forgetting the `import "dotenv/config";` line and then getting a "missing API key" error later, even though the key really is in the `.env` file — the file just was never loaded into the program.

3. **Create one shared model instance** to reuse across every lab step below:

```
const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });
```

Why `temperature: 0`: For learning labs, we want the model to behave predictably and consistently every time you run the code, rather than getting creative/random answers. `0` means "be as deterministic as possible."

Lab 4.1 — Build a `RunnableSequence`: prompt → model → parser

Goal: Build the classic three-step chain and run it.

```
const translatePrompt = ChatPromptTemplate.fromTemplate(
  "Translate the following English sentence into French. Only output the
translated sentence, nothing else.\n\nSentence: {sentence}"
);

const translateChain = translatePrompt.pipe(model).pipe(new
StringOutputParser());

const result = await translateChain.invoke({ sentence: "The weather is beautiful
today." });
console.log(result);
```

Step-by-step explanation:

- `ChatPromptTemplate.fromTemplate(...)` builds a Runnable that turns `{ sentence: "..." }` into a properly formatted chat message.
- `.pipe(model)` sends that formatted message to the chat model and gets back a raw `AIMessage` object (which contains more than just text — metadata too).
- `.pipe(new StringOutputParser())` strips that raw object down to just the plain text string you actually want.
- `.invoke({ sentence: "..." })` runs the *entire* three-step pipeline in one call, because the whole pipeline is itself a single Runnable.

Expected result: A console log of something like `Le temps est magnifique aujourd'hui.`

How to verify it worked: The output should be plain French text with no extra quotation marks, JSON formatting, or explanation — just the sentence. If you see something like `AIMessage { content: "..." }` printed instead of plain text, it means you forgot to `.pipe()` the `StringOutputParser()` at the end.

Common mistakes:

- *Forgetting `await`:* `invoke()` returns a Promise. If you forget `await`, `result` will be a `Promise` object, not the text, and `console.log(result)` will print something like `Promise { <pending> }`.
- *Mismatched variable names:* If your prompt template uses `{sentence}` but you call `.invoke({ text: "..." })`, you'll get an error saying a required input variable is missing. The keys in your `.invoke()` object must exactly match the `{placeholders}` in your prompt template.

Lab 4.2 — Build a `RunnableParallel` to generate a summary and keywords simultaneously

Goal: From one paragraph of text, get back both a one-sentence summary and a comma-separated list of keywords, generated concurrently.

```
const summaryChain = ChatPromptTemplate.fromTemplate(
  "Summarize the following text in exactly one sentence:\n\n{text}"
).pipe(model).pipe(new StringOutputParser());

const keywordChain = ChatPromptTemplate.fromTemplate(
  "List 5 important keywords from the following text, comma-separated, nothing
else:\n\n{text}"
).pipe(model).pipe(new StringOutputParser());

const parallelChain = RunnableParallel.from({
  summary: summaryChain,
  keywords: keywordChain,
});

const article = "LangChain is a framework for building applications powered by
large language models. It provides tools for prompt management, chaining calls,
connecting to external data sources, and building autonomous agents that can
reason and act.";

const output = await parallelChain.invoke({ text: article });
console.log(output);
```

Step-by-step explanation:

- We build *two separate, complete chains* (`summaryChain` and `keywordChain`), each capable of running entirely on its own.
- `RunnableParallel.from({...})` wraps both of them into a single Runnable. The **keys** of the object (`summary`, `keywords`) become the keys of the final output object.
- When you call `.invoke({ text: article })`, that same `{ text: article }` input is sent to *both* `summaryChain` and `keywordChain` at the same time.

Expected result:

```
{
  summary: "LangChain is a framework that helps developers build LLM-powered
applications through prompt management, chaining, data integration, and
agents.",
```

```
  keywords: "LangChain, framework, large language models, prompt management,
agents"
}
```

How to verify it worked: You should get a single JavaScript object with exactly two keys: `summary` (a string) and `keywords` (a string). If you only get one of them, double check you actually used `RunnableParallel.from()` and not accidentally just called one chain directly.

Common mistakes:

- Using the *same* key name in both branches (LangChain will just overwrite one with the other — you'll silently lose data, not get an error).
- Assuming parallel branches can talk to *each other*. They can't — they all start from the exact same original input and run independently; one branch's output is never visible to another branch.

Lab 4.3 — Use `RunnableBranch` to route queries to different chains by category

Goal: Build a tiny "router" that sends a question to a different specialized chain depending on its category.

```
const billingChain = ChatPromptTemplate.fromTemplate(
  "You are a billing support agent. Answer this billing question helpfully:
{question}"
).pipe(model).pipe(new StringOutputParser());

const technicalChain = ChatPromptTemplate.fromTemplate(
  "You are a technical support agent. Answer this technical question helpfully:
{question}"
).pipe(model).pipe(new StringOutputParser());

const generalChain = ChatPromptTemplate.fromTemplate(
  "You are a general support agent. Answer this question helpfully: {question}"
).pipe(model).pipe(new StringOutputParser());

const branch = RunnableBranch.from([
  [(input) => input.category === "billing", billingChain],
  [(input) => input.category === "technical", technicalChain],
  generalChain, // fallback if no condition above matched
]);

const result = await branch.invoke({
  category: "billing",
  question: "Why was I charged twice this month?",
});
console.log(result);
```

Step-by-step explanation:

- Each condition is a small array: `[conditionFunction, chainToUseIfTrue]`.
- `RunnableBranch.from([...])` checks each condition **in the order you listed them**. The first one that returns `true` "wins," and its chain handles the input. Everything else is skipped entirely.

- The very last item in the array (with no condition wrapped around it) is the fallback — it always runs if nothing above matched.

Expected result: A console log of a billing-style answer addressing the duplicate charge.

How to verify it worked: Try changing `category: "billing"` to `category: "technical"` and re-run — you should see the response noticeably change tone/role (now answering as a technical agent). Try an unrecognized category like `"random"` and confirm it falls back to `generalChain`.

Common mistakes:

- Putting the fallback chain *first* in the array by accident — since branches are checked top-to-bottom and the fallback has no condition, putting it first means it always "wins" and nothing else ever runs.
- Writing a condition function that doesn't return a strict boolean (e.g., returning the category string itself instead of a `true`/`false` comparison) — always make sure your condition function does an actual comparison like `=== "billing"`.

Lab 4.4 — Wrap a custom function with `RunnableLambda` and compose it in a pipeline

Goal: Insert your own plain JavaScript logic into a LangChain pipeline.

```
const wordCounter = RunnableLambda.from((text) => {
  const count = text.trim().split(/\s+/).length;
  return `${text}\n\n[Word count: ${count}]`;
});

const summarizeAndCount = ChatPromptTemplate.fromTemplate(
  "Summarize this text in 2-3 sentences:\n\n{text}"
)
  .pipe(model)
  .pipe(new StringOutputParser())
  .pipe(wordCounter);

const result = await summarizeAndCount.invoke({ text: article });
console.log(result);
```

Step-by-step explanation:

- `RunnableLambda.from((text) => {...})` takes a plain function (input in, output out) and turns it into something `.pipe()` can accept.
- Because the summarization chain ends with `StringOutputParser()`, by the time data reaches `wordCounter`, it's a plain string — exactly what our function expects.
- The lambda appends a word count to the end of the summary, entirely with ordinary JavaScript — no LLM call involved in this step.

Expected result: The model's summary, followed by something like `[Word count: 24]`.

How to verify it worked: Manually count the words in the printed summary and confirm the

number matches what's printed in brackets.

Common mistakes:

- Forgetting that whatever comes *before* your `RunnableLambda` determines the shape of its input. If you put `wordCounter` right after the raw `model` call (before the `StringOutputParser`), your function would receive an `AIMessage` object instead of a plain string, and `text.trim()` would throw an error because `AIMessage` objects don't have a `.trim()` method.

Lab 4.5 — Stream output token-by-token from a runnable chain

Goal: See the model's answer appear gradually, the same way ChatGPT's interface displays answers, instead of waiting for the whole response.

```
const storyChain = ChatPromptTemplate.fromTemplate(
  "Write a short, 3-sentence story about a robot learning to paint."
).pipe(model).pipe(new StringOutputParser());

const stream = await storyChain.stream({});

for await (const chunk of stream) {
  process.stdout.write(chunk);
}
console.log("\n\n--- Done streaming ---");
```

Step-by-step explanation:

- `.stream(...)` doesn't give you the final answer directly — it gives you an *async iterator*, which is a special object you loop over with `for await`.
- Each time through the loop, `chunk` is a small piece of text (sometimes a single word, sometimes a few characters) that the model has just produced.
- `process.stdout.write(chunk)` prints each piece *without* a newline, so the text appears to "type itself out" in your terminal in real time.

Expected result: You should visually see the story appear gradually in your terminal, word by word, rather than all at once.

How to verify it worked: If the text appears all at once (no visible delay/typing effect), check that you used `.stream()` and not `.invoke()` — they look similar but behave completely differently.

Common mistakes:

- Using `console.log(chunk)` instead of `process.stdout.write(chunk)` — `console.log` adds a newline after every single chunk, which makes the output an unreadable vertical list of word-fragments instead of flowing text.
- Forgetting `for await` and writing a normal `for...of` loop — since `stream()` returns an *asynchronous* iterator, a regular loop will throw a TypeError.

4.3 Knowledge Check — Unit 04

Summary

A **Runnable** is the universal interface (`invoke` / `batch` / `stream`) that every LangChain component shares, which is what allows them all to be connected together. `RunnableSequence` (or `.pipe()`) chains steps one after another. `RunnableParallel` runs multiple branches on the same input simultaneously and collects results into an object. `RunnableBranch` routes input down different paths based on conditions, checked in order, with a mandatory fallback. `RunnableLambda` lets you drop ordinary functions into a chain. `RunnablePassthrough` forwards input unchanged, which is essential when you need to preserve the original input alongside a transformed version of it later in the pipeline (you'll use this heavily in RAG, Phase 3).

Key Takeaways

- Everything in LangChain is a Runnable — that's why everything can be `.pipe()`-d together.
- `RunnableSequence` = "do this, then this, then this" (sequential).
- `RunnableParallel` = "do all of these at once, from the same starting input" (concurrent, fan-out).
- `RunnableBranch` = "decide which single path to take" (conditional, like an if/else chain).
- `RunnableLambda` = "let plain code participate in the pipeline."
- `RunnablePassthrough` = "keep a copy of the original input available downstream."
- `.pipe()` is just readable shorthand for `RunnableSequence.from([...])`.

Practice Questions

1. What three methods does every Runnable expose, and what does each one do differently?
2. You want to generate a translation AND a sentiment score from the same sentence at the same time. Which Runnable type should you use, and why?
3. In `RunnableBranch`, what happens if two of your conditions could both technically be `true` for the same input?
4. Why might you need `RunnablePassthrough` even though it "does nothing" to the data?
5. What's the practical difference between calling `.invoke()` and `.stream()` on the same chain?

Exercises

1. Modify Lab 4.1 so the chain translates into **Spanish** instead of French, without changing anything except the prompt text.
2. Modify Lab 4.2 so the `RunnableParallel` produces **three** outputs instead of two: `summary`, `keywords`, and `sentiment` (positive/negative/neutral).

3. Modify Lab 4.3 so there's a third explicit category, `"sales"`, with its own chain, in addition to the existing fallback.
4. Write your own `RunnableLambda` that takes a string and returns it reversed, and insert it at the end of any chain from this unit.

Mini Project — "Smart Router" Console App

Build a small Node.js script that:

1. Accepts a `category` (`"billing"`, `"technical"`, `"general"`) and a `question` as input (you can hard-code a few test cases for now — you'll learn to take real user input in a later phase).
2. Uses `RunnableBranch` to route to the right specialized chain.
3. Uses `RunnableParallel` to *also* generate a one-word urgency rating (`"low"`, `"medium"`, `"high"`) for the question, alongside the actual answer.
4. Streams the final chosen answer to the console using `.stream()`.

This mini project forces you to combine routing, parallel execution, and streaming in one place — exactly the kind of composition you'll do constantly in real LangChain applications.

Unit 05 — Chains

Phase: Core | **Difficulty:** Intermediate

Why this unit matters

Unit 04 taught you the individual Lego bricks (Runnables). Unit 05 is about **patterns** — the common shapes that real applications need, built using those bricks. "Chain" is a slightly older, more general LangChain term for "a pipeline of steps," and you'll see it used both as a formal class name and as a general concept. By the end of this unit you should be able to look at *any* multi-step LLM task and immediately picture which chain "shape" (simple, parallel, conditional, or nested) fits it.

5.1 Concept and Theory

5.1.1 Chains as data-flow pipelines

The simple explanation: A chain is simply: **input → some processing → output**, where "processing" might be one step or many steps. You already built chains in Unit 04 without necessarily calling them that — `prompt.pipe(model).pipe(parser)` *is* a chain.

Why this framing matters: Thinking of your application as a *flow of data* (rather than as "a bunch of function calls") makes it much easier to design. Before writing any code, you should be able to sketch your chain on paper as boxes and arrows: "user input" → box 1 → box 2 → "final output." If

you can draw it, you can build it with Runnables.

5.1.2 Simple chain: prompt + model

The simple explanation: This is the smallest possible chain — just two steps. You format a prompt, then send it straight to a model. You won't always need a parser if you're fine working with the raw `AIMessage` object, but in practice almost every chain includes one.

Analogy: This is the "ask a single question, get a single answer" pattern — like a vending machine. You put in a coin (your prompt), it processes internally, and a snack (the model's answer) comes out. No branching, no extra steps.

5.1.3 Parallel chains — multiple outputs from one input

The simple explanation: This is exactly the `RunnableParallel` pattern from Unit 04, viewed from the "what problem does it solve" angle: you have *one* piece of input data but need *several different transformations* of it, all at once.

When to use it: Whenever your application's output is naturally "a report with multiple sections" rather than "a single answer" — for example, a document analysis tool that returns a title, a summary, AND a list of action items, all from the same uploaded document.

5.1.4 Conditional chains — branching logic based on previous output

The simple explanation: This is the `RunnableBranch` pattern, but used specifically when the *condition itself* depends on something the chain has already figured out earlier — not just raw user input. For example: first classify a customer message into a category using the LLM itself, *then* branch based on that LLM-produced classification.

Why this is more powerful than a simple branch: In Lab 4.3, you (the developer) decided the category in advance. In a real conditional chain, you often don't know the category ahead of time — you ask the LLM to figure it out as step one, and then branch on *that* result as step two. This means your chain has to be a `RunnableSequence` where step one is "classify," and step two is a `RunnableBranch` that reads the classification.

5.1.5 Nesting chains within chains for multi-stage workflows

The simple explanation: Because every chain is itself a single Runnable, you can use a *whole finished chain* as just one ingredient inside an even bigger chain. This is the "Lego brick" idea from Unit 04 taken to its logical conclusion.

Analogy: A car engine is itself made of smaller assemblies (the fuel system, the cooling system, the ignition system), each of which is, in turn, made of even smaller parts. You don't need to think about every individual bolt every time you think about "the engine" — you think in terms of completed sub-assemblies. Nested chains let you do the same thing in code: build and test a "summarization chain" once, then just treat it as one black-box step inside a bigger "full document analysis chain."

5.2 Practical Lab

Lab 5.1 — Build a simple translation chain (English → French)

Goal: Recreate the simplest possible chain shape from scratch, paying attention to *why* each piece is there.

```
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

const translationPrompt = ChatPromptTemplate.fromTemplate(
  "You are a professional translator. Translate the following English text to
French. Output only the translation.\n\nText: {text}"
);

const simpleChain = translationPrompt.pipe(model).pipe(new
StringOutputParser());

console.log(await simpleChain.invoke({ text: "Where is the nearest train
station?" }));
```

Why each step exists:

- The prompt's instruction ("Output only the translation") exists because, without it, the model often adds chatty extras like *"Sure! Here's the translation: ..."*, which would break any downstream code expecting a clean string.
- `temperature: 0` keeps translations consistent across repeated runs — useful for testing.

Expected result: `Où est la gare la plus proche ?`

How to verify it worked: The output should contain *only* the French sentence, with correct punctuation, and no English commentary wrapped around it.

Common mistake: Skipping the "output only the translation" instruction and then writing parsing code that breaks the moment the model adds an introductory sentence. Prompt instructions are often the *easiest* fix for messy output — easier than complex parsing logic.

Lab 5.2 — Build a parallel chain: generate a title and a summary from one document

```
import { RunnableParallel } from "@langchain/core/runnables";

const titleChain = ChatPromptTemplate.fromTemplate(
  "Write one short, catchy title (under 10 words) for this text:\n\n{document}"
).pipe(model).pipe(new StringOutputParser());

const summaryChain = ChatPromptTemplate.fromTemplate(
  "Summarize this text in 2 sentences:\n\n{document}"
).pipe(model).pipe(new StringOutputParser());

const docAnalysisChain = RunnableParallel.from({
  title: titleChain,
  summary: summaryChain,
```

```
});

const document = "Remote work has reshaped how companies think about office
space. Many organizations now operate with hybrid schedules, offering employees
flexibility while maintaining a smaller physical footprint. Critics argue this
weakens company culture, while supporters point to higher employee satisfaction
and lower overhead costs.";

console.log(await docAnalysisChain.invoke({ document }));
```

Step-by-step explanation: Same `RunnableParallel` mechanics as Lab 4.2, but now framed as a real use case: a "document analysis" feature. Notice both prompt templates use the *same* placeholder name, `{document}` — this matters because `RunnableParallel` sends the exact same `{ document: "..." }` object to every branch; if one branch's template expected a *different* variable name, it would throw a "missing input variable" error.

Expected result: `{ title: "...", summary: "..." }` — a short, punchy title and a two-sentence summary, both about the same input text.

Verification tip: Read the title and summary back-to-back — they should clearly be "about the same thing," just at different levels of detail/style.

Lab 5.3 — Build a conditional chain that routes based on question category

Goal: This time, let the LLM itself *decide* the category first, then branch — the more realistic version of Lab 4.3.

```
import { RunnableSequence, RunnableBranch } from "@langchain/core/runnables";
import { RunnableLambda } from "@langchain/core/runnables";

// Step 1: classify the incoming question
const classifyPrompt = ChatPromptTemplate.fromTemplate(
  `Classify this customer question into exactly one word: "billing",
"technical", or "general". Respond with only that one word.\n\nQuestion:
{question}`
);
const classifyChain = classifyPrompt.pipe(model).pipe(new StringOutputParser());

// Step 2: each specialized responder
const billingChain = ChatPromptTemplate.fromTemplate(
  "Answer this billing question helpfully and concisely: {question}"
).pipe(model).pipe(new StringOutputParser());

const technicalChain = ChatPromptTemplate.fromTemplate(
  "Answer this technical support question helpfully and concisely: {question}"
).pipe(model).pipe(new StringOutputParser());

const generalChain = ChatPromptTemplate.fromTemplate(
  "Answer this general question helpfully and concisely: {question}"
).pipe(model).pipe(new StringOutputParser());

const responderBranch = RunnableBranch.from([
  [(input) => input.category.trim().toLowerCase() === "billing", billingChain],
  [(input) => input.category.trim().toLowerCase() === "technical",
technicalChain],
  generalChain,
```

```
]);

// Step 3: wire it all into one full pipeline
const fullChain = RunnableSequence.from([
  // First, attach a "category" field by running the classifier,
  // while keeping the original question available too.
  RunnableLambda.from(async (input) => {
    const category = await classifyChain.invoke({ question: input.question });
    return { question: input.question, category };
  }),
  responderBranch,
]);

console.log(await fullChain.invoke({ question: "My invoice shows a charge I
don't recognize." }));
```

Step-by-step explanation:

- This chain has **two stages**: stage one classifies, stage two responds.
- We use a `RunnableLambda` as "glue" to call the classifier chain manually and then re-attach its result onto the original input object (so we don't lose the original `question` text while adding the new `category` field) — this is a manual alternative to `RunnablePassthrough` + `RunnableParallel`, and it's worth seeing both styles.
- `responderBranch` then reads `input.category` (produced by stage one) to decide where to route.

Expected result: The classifier should output `"billing"`, and the final answer should come from `billingChain`, addressing the unrecognized charge.

How to verify it worked: Temporarily add `console.log("Classified as:", category)` inside the lambda to see the LLM's classification before the branch consumes it. This kind of "print what's happening at each stage" debugging is extremely valuable while learning multi-stage chains.

Common mistakes:

- Trusting the LLM's classification output blindly. LLMs occasionally output extra whitespace, punctuation, or capitalization variants (`"Billing"` vs `"billing"`) — that's exactly why the branch conditions use `.trim().toLowerCase()` for safety.
- Forgetting to `await` inside a `RunnableLambda` when the function itself needs to call another chain (since `classifyChain.invoke()` is asynchronous).

Lab 5.4 — Nest two chains into a multi-stage pipeline

Goal: Treat `docAnalysisChain` from Lab 5.2 as a single black-box ingredient inside a bigger chain that also generates a recommended social-media post based on the title and summary.

```
const socialPostChain = ChatPromptTemplate.fromTemplate(
  `Write a short, engaging social media post (under 280 characters) promoting an
article.\nTitle: {title}\nSummary: {summary}`
).pipe(model).pipe(new StringOutputParser());
```

```
const fullPipeline = RunnableSequence.from([
  docAnalysisChain,     // produces { title, summary }
  socialPostChain,      // consumes { title, summary } directly
]);

console.log(await fullPipeline.invoke({ document }));
```

Step-by-step explanation: This is the "nesting" idea in its purest form. `docAnalysisChain` is a complete, independently-testable chain (you already verified it works in Lab 5.2). We're not rewriting any of its internals — we're just placing the *finished* chain as step one of a new, bigger `RunnableSequence`. Its output shape, `{ title, summary }`, conveniently matches exactly what `socialPostChain`'s prompt template expects as input variables, so no glue code is even needed between them.

Expected result: A short, punchy social-media-style post referencing both the article's title and its summary content.

How to verify it worked: Check that the final post is under 280 characters and clearly references concepts from the original document (not generic filler text).

Common mistake: Forgetting that the *output shape* of one chain must match the *expected input shape* of the next chain when nesting them directly in a `RunnableSequence`. If `docAnalysisChain` had been named with different keys (say, `headline` instead of `title`), `socialPostChain`'s `{title}` placeholder would throw a missing-variable error, and you'd need a `RunnableLambda` in between to rename the fields.

5.3 Knowledge Check — Unit 05

Summary

A "chain" is any pipeline that turns input into output — the term covers everything from the simplest two-step `prompt → model` chain up to large, multi-stage pipelines made of nested sub-chains. The four shapes you now know are: **simple** (one path), **parallel** (multiple simultaneous outputs from the same input), **conditional** (one path chosen dynamically, often based on a classification step that runs *earlier in the same chain*), and **nested** (a complete chain reused as a single step inside a bigger chain). Real applications almost always combine more than one of these shapes together.

Key Takeaways

- "Chain" and "Runnable pipeline" are essentially the same idea — Unit 04 taught you the parts, Unit 05 teaches you the recipes.
- A conditional chain is usually *two* stages: classify, then branch — not just a branch alone.
- Nesting works because a finished chain is, itself, just another Runnable — output shape must match the next step's expected input shape.
- Always design your chain shape on paper (boxes and arrows) before writing code.

Practice Questions

1. What's the difference between the `RunnableBranch` you built in Unit 04 (Lab 4.3) and the conditional chain you built in Unit 05 (Lab 5.3)?
2. Why did `socialPostChain` not need any glue code to consume `docAnalysisChain`'s output directly?
3. Give a real-world example (not from this guide) of a task that would naturally use a parallel chain.
4. Why is it good practice to test each sub-chain (like `docAnalysisChain`) on its own *before* nesting it into a bigger pipeline?

Exercises

1. Extend Lab 5.3 to support a fourth category, `"sales"`, with its own specialized prompt.
2. Extend Lab 5.4 so the final pipeline also produces an email subject line (a third parallel output alongside the social post), reusing `docAnalysisChain`'s output.
3. Rebuild Lab 5.1's translation chain so it works for *any* target language, by adding a `{targetLanguage}` variable to the prompt template instead of hardcoding "French."

Mini Project — "Content Repurposing Pipeline"

Build a chain that takes one long-form article and nests together: (1) a parallel stage producing a title, summary, and 5 keywords; (2) a conditional stage that picks a different "voice" (formal / casual / humorous) for the final output based on a `tone` field you pass in; (3) a final nested stage that turns the chosen-tone summary into a ready-to-post social media caption. This combines every chain shape from this unit into one realistic content-creation tool.

Unit 06 — Memory

Phase: Core | **Difficulty:** Intermediate

Why this unit matters

Every chain you've built so far has **no memory** — call it twice in a row, and the second call has no idea the first call ever happened. That's fine for a translator or summarizer, but it's a dealbreaker for a chatbot. Unit 06 teaches you how to give your chains a "remembered" conversation, which is the foundation of every chatbot-style application you'll build going forward (including the conversational RAG you'll meet in Phase 3).

6.1 Concept and Theory

6.1.1 Why LLM API calls are stateless by default

The simple explanation: Every single call to a chat model's API is completely independent — the

model has *zero* built-in memory of any previous call you made, even one millisecond earlier. Each request is like meeting a stranger who has never seen you before and showing them only the current page of a book, with no idea what happened on previous pages.

Why is it built this way? This is actually a deliberate, sensible design choice, not a limitation that "should" be fixed at the API level. If the API silently remembered every conversation from every user automatically, it would be a privacy nightmare (your conversation could leak into someone else's session), it would be impossible to scale (the server would need to store unlimited history for every user forever), and you'd have no control over what "history" actually means for your specific application. Instead, **the API treats every single call as a fresh blank page, and it's the developer's job to hand the model whatever earlier context it should "remember"** by including it directly in the new request.

The practical implication: "Memory" in LangChain isn't magic — it's simply *automated bookkeeping* that stores previous messages somewhere (in a variable, a file, a database) and then re-inserts that stored history into each new prompt before sending it to the model. The model itself never "remembers" anything between calls; your application remembers, and re-tells the model everything relevant each time.

6.1.2 ConversationBufferMemory — full message history storage

The simple explanation: This is the simplest possible memory: store *every single message* (both yours and the AI's) from the start of the conversation, and replay the entire transcript back to the model on every new turn.

Analogy: Imagine handing someone the *complete* written transcript of your conversation every single time you ask them a new question, so they can re-read the whole thing from the beginning before answering. It works, but the transcript gets longer and longer, and eventually it becomes expensive (in tokens, which cost money) and slow to re-read every time.

When to use it: Short conversations, prototypes, or situations where you genuinely need the model to recall details from very early in the chat.

6.1.3 ConversationBufferWindowMemory — last N messages only

The simple explanation: Instead of keeping the *entire* history forever, this keeps only the most recent N messages (you choose N), and automatically drops older ones as new ones come in.

Analogy: This is like a small whiteboard that can only fit the last few notes — when you write a new note and the board is full, the oldest note gets erased to make room.

Why does it exist? Sending the *entire* conversation history on every turn gets expensive and slow as conversations grow longer (more tokens = more cost, more processing time, and eventually you'll hit the model's maximum context length entirely). Window memory caps the cost by trading away long-term recall for predictable, bounded request sizes.

Trade-off to understand clearly: If your conversation goes 20 turns deep and your window only keeps the last 4 messages, the model will have **completely forgotten** anything said in turn 1 — it's not "fuzzy" memory, it's a hard cutoff.

6.1.4 ConversationSummaryMemory — summarized rolling history

The simple explanation: Instead of keeping exact messages OR a hard cutoff, this approach periodically asks the LLM itself to *compress* the conversation so far into a short running summary, and that summary (not the raw transcript) is what gets sent on future turns.

Analogy: Think of a meeting where, instead of replaying the full audio recording every time someone joins late, a secretary keeps a single, continuously-updated one-paragraph summary of "what's happened in this meeting so far." New attendees just read that paragraph instead of the entire transcript.

Why does it exist? This is a middle ground: unlike window memory, it can preserve the *gist* of something said 50 messages ago; unlike full buffer memory, it stays compact regardless of how long the conversation runs, because the summary itself stays roughly the same length even as more turns happen (older detail naturally gets compressed away or dropped in favor of newer detail).

Trade-off to understand clearly: Summarization costs an *extra* LLM call every time it updates the summary, and summaries can lose precise details (like an exact number or a specific quoted sentence) that a full buffer would have preserved.

6.1.5 Custom Memory — storing specific user data or preferences

The simple explanation: Sometimes you don't want to remember the whole conversation at all — you just want to remember a few *specific facts* (the user's name, their preferred language, a setting they chose) and inject just those facts into every prompt, regardless of how the rest of the conversation has flowed.

Analogy: A good waiter at a restaurant you visit regularly doesn't necessarily remember your entire life story, but they *do* remember "this customer prefers their steak medium-rare and doesn't like cilantro" — a small, targeted set of facts that gets applied to every future visit.

Why does it exist? Built-in memory types are conversation-shaped (a list of messages). Custom memory is for application-shaped state — structured facts that don't naturally look like a chat transcript, but that you still want available to the model every time.

6.1.6 Attaching memory to a chain or runnable

The simple explanation: In LangChain.js (LCEL style), "attaching memory" usually means: (1) keep your message history in a variable or store somewhere in your own code, (2) include a `MessagesPlaceholder` in your `ChatPromptTemplate` (you met this in Unit 02) as the spot where history gets inserted, and (3) before each `.invoke()`, load the current history into that placeholder's variable, and after each `.invoke()`, save the new exchange back into your history store.

Why it's done this way in modern LangChain.js: Older versions of LangChain had dedicated `Memory` *classes* baked invisibly into chains. The modern LCEL approach makes memory more explicit and transparent: *you* control exactly what history looks like and where it's stored, and the chain itself stays a simple, predictable Runnable. This is more code to write, but it's much easier to understand, debug, and customize.

6.2 Practical Lab

Lab 6.1 — Build a chatbot that remembers messages with buffer-style memory

Goal: Build a multi-turn chatbot where the second message correctly references something said in the first message.

```
import { ChatPromptTemplate, MessagesPlaceholder } from
"@langchain/core/prompts";
import { HumanMessage, AIMessage } from "@langchain/core/messages";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

const chatPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a friendly, helpful assistant."],
  new MessagesPlaceholder("history"),
  ["human", "{input}"],
]);

const chatChain = chatPrompt.pipe(model).pipe(new StringOutputParser());

// This array IS our "buffer memory" — a plain list of every message so far.
let history = [];

async function chat(userInput) {
  const response = await chatChain.invoke({ history, input: userInput });
  // Save BOTH sides of this exchange into the buffer for next time.
  history.push(new HumanMessage(userInput));
  history.push(new AIMessage(response));
  return response;
}

console.log(await chat("My name is Aarav and I'm building a resume-writing
tool."));
console.log(await chat("What did I say my name was?"));
```

Step-by-step explanation:

- `MessagesPlaceholder("history")` is a special spot inside the prompt template that gets filled in with a *list of message objects*, not a plain string — this is exactly how you "replay the transcript" to the model.
- `history` is a plain JavaScript array acting as our buffer memory — nothing fancy, just objects representing who said what.
- Every time `chat()` runs, it (1) sends the *current* history plus the new input, then (2) appends both the new human message and the new AI response onto `history`, so the *next* call will include this exchange too.

Expected result: The second call should correctly answer `"Your name is Aarav."` — proof that the model "remembered," even though, as you now know, it didn't actually remember anything itself; your code re-sent the full transcript.

How to verify it worked: Add a `console.log(history)` right before the second `chat()` call and confirm it contains both the `HumanMessage` and `AIMessage` from the first exchange.

Common mistakes:

- Forgetting to push *both* sides of the exchange (only saving the human message, but not the AI's reply, or vice versa) — this creates a lopsided, confusing transcript.
- Re-creating an empty `history = []` array inside the `chat()` function itself (instead of keeping it outside, in the enclosing scope) — this would silently reset memory on every single call, defeating the entire purpose.

Lab 6.2 — Switch to window-style memory and observe context dropping beyond N turns

Goal: See exactly what happens to memory once it exceeds your chosen limit.

```
const WINDOW_SIZE = 2; // keep only the last 2 *exchanges* (4 messages: 2 human
+ 2 AI)

let windowHistory = [];

async function chatWithWindow(userInput) {
  const response = await chatChain.invoke({ history: windowHistory, input:
userInput });

  windowHistory.push(new HumanMessage(userInput));
  windowHistory.push(new AIMessage(response));

  // Keep only the most recent WINDOW_SIZE * 2 messages (human + AI pairs).
  if (windowHistory.length > WINDOW_SIZE * 2) {
    windowHistory = windowHistory.slice(-WINDOW_SIZE * 2);
  }

  return response;
}

console.log(await chatWithWindow("My favorite color is teal."));
console.log(await chatWithWindow("I live in Bijnor."));
console.log(await chatWithWindow("I work as a resume writer."));
console.log(await chatWithWindow("What is my favorite color?")); // should now
be FORGOTTEN
```

Step-by-step explanation:

- After every exchange, we check whether `windowHistory` has grown past our limit, and if so, we `.slice(-WINDOW_SIZE * 2)` to keep only the most recent messages, discarding everything older.
- By the time we ask "What is my favorite color?" — the 4th message — the very first exchange (about the favorite color) has already been pushed out of the window by the two exchanges that came after it.

Expected result: The model should respond that it doesn't know your favorite color (or guess incorrectly), because that information has genuinely been deleted from what gets sent to it.

How to verify it worked: Print `windowHistory.length` right before the final call — it should be exactly 4 (2 exchanges × 2 messages each), and the favorite-color exchange should not be among them if you log the contents.

Common mistake: Setting `WINDOW_SIZE` to count *messages* in your head but implementing it as *exchanges* (or vice versa) — be explicit and consistent about whether your "N" means individual messages or full human+AI pairs, since mixing the two up is a very common off-by-one bug.

Lab 6.3 — Implement summary-style memory and inspect the generated summary

Goal: Instead of keeping raw messages, maintain a single rolling summary that updates after each turn.

```
const summaryPrompt = ChatPromptTemplate.fromTemplate(
  `Here is the current summary of a conversation so far:\n{currentSummary}
\n\nNew exchange:\nHuman: {humanInput}\nAI: {aiResponse}\n\nUpdate the summary
to naturally include this new exchange. Keep it concise (3-4 sentences max).`
);
const summaryUpdateChain = summaryPrompt.pipe(model).pipe(new
StringOutputParser());

let runningSummary = "No conversation has happened yet.";

async function chatWithSummary(userInput) {
  // Use the summary as a single system-style note instead of a full message
list.
  const summaryAwarePrompt = ChatPromptTemplate.fromMessages([
    ["system", "You are a helpful assistant. Here is a summary of the
conversation so far: {summary}"],
    ["human", "{input}"],
  ]);
  const chain = summaryAwarePrompt.pipe(model).pipe(new StringOutputParser());

  const response = await chain.invoke({ summary: runningSummary, input:
userInput });

  // Update the rolling summary to fold in this new exchange.
  runningSummary = await summaryUpdateChain.invoke({
    currentSummary: runningSummary,
    humanInput: userInput,
    aiResponse: response,
  });

  return response;
}

await chatWithSummary("I'm planning a trip to Manali next month.");
await chatWithSummary("I'd like recommendations for budget hotels there.");
console.log("Current summary:", runningSummary);
```

Step-by-step explanation:

- We maintain **one string**, `runningSummary`, instead of a growing array.
- Every turn does *two* model calls: one to actually answer the user (using the summary as background context), and a second to *update* the summary so it now reflects this newest exchange too.
- Over many turns, `runningSummary` stays roughly the same length (a few sentences), no matter how long the actual conversation gets — that's the entire point.

Expected result: After two exchanges, `runningSummary` should read something like: *"The user is planning a trip to Manali next month and asked for budget hotel recommendations there."*

How to verify it worked: Confirm the summary mentions *both* pieces of information (Manali trip, budget hotel request) even though they were said in two separate messages — that's proof the summary update step is correctly folding new information into the old.

Common mistake: Forgetting that summary memory costs **double** the API calls per turn (one to answer, one to re-summarize) — beginners are often surprised when their token usage or latency is higher than expected with this approach, compared to plain buffer memory.

Lab 6.4 — Build custom memory that stores a user's name and topic preferences

Goal: Store a small, structured set of facts (not a transcript) and inject them into every prompt.

```
// A plain object representing "everything we know about this user" —
// this is our entire custom memory. No message list required.
const userProfile = {
  name: null,
  preferredTopics: [],
};

const profileAwarePrompt = ChatPromptTemplate.fromTemplate(
  `You are a helpful assistant. Known facts about the user: name = {name},
interested in = {topics}.\nUse these facts naturally if relevant, but do not
mention that you were "told" them.\n\nUser: {input}`
);
const profileChain = profileAwarePrompt.pipe(model).pipe(new
StringOutputParser());

function updateProfileFromInput(input) {
  // Extremely simple rule-based extraction for teaching purposes.
  // (In a real app you might use the LLM itself, via withStructuredOutput, to
extract these facts.)
  const nameMatch = input.match(/my name is (\w+)/i);
  if (nameMatch) userProfile.name = nameMatch[1];

  if (/langchain|ai|machine learning/i.test(input)) {
    if (!userProfile.preferredTopics.includes("AI/LangChain")) {
      userProfile.preferredTopics.push("AI/LangChain");
    }
  }
}

async function chatWithProfile(userInput) {
  updateProfileFromInput(userInput);
  return profileChain.invoke({
    name: userProfile.name ?? "unknown",
    topics: userProfile.preferredTopics.join(", ") || "none specified yet",
    input: userInput,
  });
}

console.log(await chatWithProfile("My name is Winger and I'm learning
LangChain."));
console.log(await chatWithProfile("Can you recommend what I should study
next?"));
```

Step-by-step explanation:

- `userProfile` is a plain object — our entire "memory" — that has nothing to do with a list of messages.
- `updateProfileFromInput` is simple pattern-matching code (not an LLM call) that pulls out a name and a topic interest from the user's raw text.
- Every prompt explicitly includes `{name}` and `{topics}`, so the model can naturally reference them — notice the instruction *"do not mention that you were told them"* exists so the bot's replies sound natural ("Since you're into AI/LangChain, I'd suggest...") rather than robotic ("As you previously told me, your name is...").

Expected result: The second response should reference LangChain/AI topics specifically (because `preferredTopics` now contains `"AI/LangChain"`), even though the second message itself never mentioned LangChain again.

How to verify it worked: `console.log(userProfile)` after the first call and confirm `{ name: "Winger", preferredTopics: ["AI/LangChain"] }` is correctly populated before the second call ever runs.

Common mistake: Relying on fragile regular expressions for real production extraction (as we did here, for simplicity). In a real application, you'd typically use `model.withStructuredOutput(zodSchema)` (from Unit 01) to reliably extract structured facts from free-form user text instead of brittle regex patterns.

6.3 Knowledge Check — Unit 06

Summary

LLM API calls are stateless — the model never remembers anything between calls, so "memory" in LangChain is really just your own application re-sending relevant context on every turn. **Buffer memory** keeps the full transcript (simple, but grows expensive). **Window memory** keeps only the last N messages (bounded cost, but hard forgetting beyond the window). **Summary memory** keeps a constantly-updated short summary instead of raw messages (bounded cost *and* long-term gist retention, at the price of an extra LLM call per turn and loss of exact detail). **Custom memory** stores specific structured facts rather than a conversation transcript, which is ideal when you only care about a handful of durable facts (name, preferences, settings) rather than the flow of the conversation itself. In modern LangChain.js, memory is attached explicitly via a `MessagesPlaceholder` (for message-list-style memory) or by directly templating stored values into the prompt (for custom/summary memory) — you are always in full control of what the model "remembers."

Key Takeaways

- Statelessness is a deliberate design choice, not a flaw — your application is responsible for reconstructing relevant context every time.
- Buffer = everything, Window = last N only, Summary = compressed gist, Custom = specific

facts.

- There's always a trade-off between **completeness of recall**, **cost/speed**, and **complexity** — choose based on what your specific application actually needs to remember.
- `MessagesPlaceholder` is the standard slot in a `ChatPromptTemplate` where message-list memory gets inserted.

Practice Questions

1. If a chatbot conversation is 200 messages long, what's the main downside of using plain buffer memory the whole way through?
2. Why does summary memory require *two* model calls per turn instead of one?
3. Give an example of information that would be better suited to custom memory than to any message-history-based memory type.
4. What does it mean, technically, for the underlying LLM API to be "stateless," and who is actually responsible for simulating memory?

Exercises

1. Modify Lab 6.2 so `WINDOW_SIZE` is `4` instead of `2`, and verify the favorite-color fact survives one extra exchange longer before being forgotten.
2. Modify Lab 6.4 to also extract and remember a user's stated *experience level* (beginner/intermediate/advanced) from phrases like "I'm new to this" or "I'm experienced with...", and have the assistant adjust its tone for beginners vs. experts accordingly.
3. Combine buffer memory (Lab 6.1) with custom memory (Lab 6.4) in a single chatbot: keep the last 6 raw messages AND a small structured `userProfile` object, both injected into the same prompt.

Mini Project — "Long-Term Companion Bot"

Build a console chatbot that uses **summary memory** for general conversational flow (so it can run for many turns without unbounded cost) **plus custom memory** for a small set of durable facts (name, role, and one ongoing project they mention). After roughly 6-8 exchanges covering different topics, ask the bot a question that requires it to recall something from very early in the conversation, and confirm the summary successfully preserved it. This project forces you to combine two different memory strategies the way a real production chatbot often does.

Phase 2 Capstone Project — "Multi-Stage Customer Support Assistant"

Now that you've completed Units 04-06, build one project that uses **everything** from this phase together. This is meant to feel like a small, real application — take your time with it.

Requirements:

1. **Conversation memory:** Use window-style memory (last 6 messages) so the assistant remembers recent context in a multi-turn conversation, without unbounded cost.
2. **Classification + routing:** On every new user message, first classify it (billing / technical / general / sales) using an LLM call, then route to a specialized prompt for that category using `RunnableBranch` — exactly like Lab 5.3, but now memory-aware (the classification and the specialized response should both have access to recent conversation history via `MessagesPlaceholder`).
3. **Parallel side-output:** Alongside the actual response, run a second `RunnableParallel` branch that produces a one-word urgency rating (`low` / `medium` / `high`) for *every* message, so a (hypothetical) human supervisor could later sort tickets by urgency.
4. **Custom memory:** Track and remember the customer's name and the single most recent product/order they've mentioned, and reference it naturally in responses once known.
5. **Streaming:** The final chosen response (not the urgency rating or classification) should be streamed to the console, not printed all at once.

Suggested structure to build it in:

- Step 1: Build the classification + custom-memory-fact-extraction step.
- Step 2: Build the four specialized response chains (billing/technical/general/sales), each accepting `{ history, input, customerName }`.
- Step 3: Wire classification → `RunnableBranch` using `RunnableSequence`.
- Step 4: Wrap the final response chain together with the urgency-rating chain inside a `RunnableParallel`, so you get back `{ response: <streaming text>, urgency: "..." }` per turn — note that you'll typically `.invoke()` the urgency branch separately from `.stream()`-ing the response branch, since you can't easily stream *and* get a clean parallel object back from the same call; this is a great moment to think carefully about which part of your pipeline truly needs to stream and which doesn't.
- Step 5: Run a 5-6 message test conversation that touches at least two different categories, and confirm: routing changes correctly, urgency ratings make sense, the customer's name is remembered and naturally referenced after being mentioned once, and old messages beyond the window are correctly forgotten.

Why this capstone matters: This single project intentionally forces you to make real architectural decisions — like "should classification share the same memory as the response?" and "what happens if window memory drops the message where the customer first gave their name?" — that don't have one single textbook answer. Wrestling with those decisions now, on a small project, is exactly what prepares you for Phase 3 (RAG Pipeline), where you'll combine these same building blocks with retrieval over real documents.

End of Phase 2. You now understand the core "grammar" of LangChain (Runnables/LCEL), the common shapes pipelines take (Chains), and how to give your applications memory. Phase 3 — RAG Pipeline — builds directly on all three of these units, since RAG is, structurally, just a chain

with a retrieval step wired in using the exact same `RunnableParallel` + `RunnablePassthrough` pattern you practiced in Unit 04.